

ECE 220 Computer Systems & Programming

Lecture 13 — Problem Solving with Recursion; Recursion with Backtracking

October 8, 2026

Prof. Sayan Mitra mitras@illinois.edu

Course website: courses.grainger.illinois.edu/ece220/fa2026

I ILLINOIS

Electrical & Computer Engineering

GRAINGER COLLEGE OF ENGINEERING

The Dead-End Problem

Place 4 queens on a 4×4 board so that no two share a row, a column, or a diagonal.

	0	1	2	3	
0	Q				— row 0: queen in column 0
1			Q		— row 1: queen in column 2
2	×	×	×	×	— row 2: every square is attacked
3					

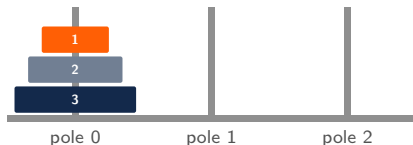
Today's Two Questions

A recursive function needs a smaller problem of the same kind, and a base case.

1. A puzzle states no recurrence. How do we find the smaller problem inside it?
2. A recursive function makes a choice and reaches a dead end. How does it go back and choose again?

Question 1 is Towers of Hanoi. Question 2 is the 4-queens board; the method is recursion with **backtracking**.

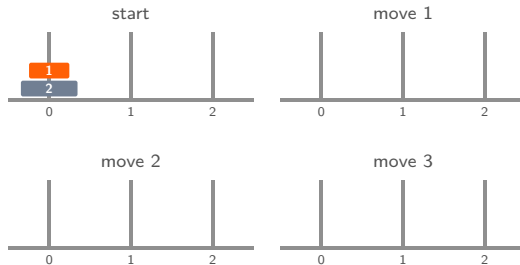
Towers of Hanoi



Move the tower of n discs from pole 0 to pole 2.

- One disc moves at a time: the top disc of one pole goes to the top of another pole
- A disc never rests on a smaller disc

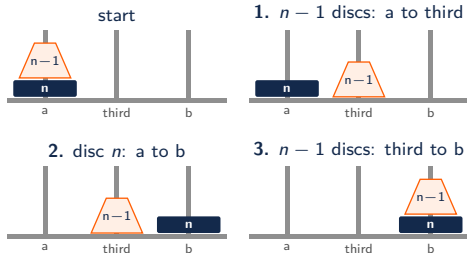
By hand, $n = 2$. Draw the poles after each move:



Fewest moves for $n = 1, 2, 3$: _____, _____, _____

In-Class Problem: movetower

`movetower(n, a, b)`: n discs from pole a to pole b



Steps 1 and 3: this problem again, for _____ discs

The third pole, from a and b : _____

Base case: _____

```
void movetower(int n, int a, int b) {
    /* base case: a tower of one disc */

    /* top n-1 discs: pole a to third pole */

    /* disc n: pole a to pole b */

    /* the n-1 discs: third pole to pole b */

}
```

Counting the Moves

`hanoi.c`, run with 3 discs:

```
Number of discs: 3
Move tower of 3 from pole 0 to pole 2
disc 1: 0 -> 2
disc 2: 0 -> 1
disc 1: 2 -> 1
disc 3: 0 -> 2
disc 1: 1 -> 0
disc 2: 1 -> 2
disc 1: 0 -> 2
```

$T(n)$ = number of moves for n discs

n	1	2	3	4	10
$T(n)$	1	3	7	_____	_____

- $T(1) = 1$ and $T(n) = \underline{\hspace{2cm}}$
- In closed form, $T(n) = \underline{\hspace{2cm}}$
- Most `movetower` records on the run-time stack at one time: _____

Every move of `movetower` is final. In the next problem a choice can turn out wrong, and the function has to take it back.

n-Queens: One Row at a Time

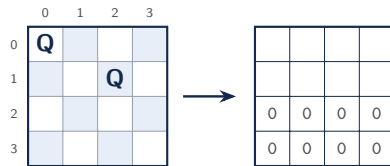
Problem: place N queens on an $N \times N$ board, no two in the same row, column, or diagonal.

1. Start at row 0. Look for a safe column, from left to right, and place a queen there.
2. Found a safe column: make a **recursive call** to place a queen in the next row.
3. Found none: **backtrack**. Return to the previous row, remove its queen, and continue with the next column there.

To **backtrack** is to undo the most recent choice and try the next alternative.

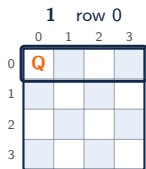
```
int board[N][N];
```

1 = queen, 0 = empty square



Fill in rows 0 and 1 of board.

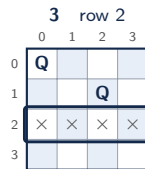
4 Queens by Hand: Queen at (0, 0)



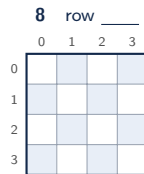
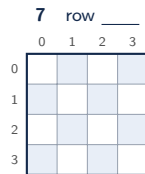
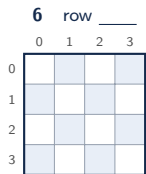
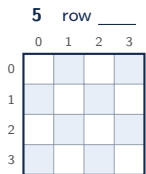
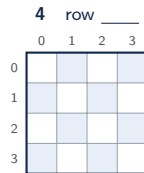
column 0 is safe:
queen placed



columns 0, 1 attacked;
queen in column 2



no safe column:
nqueen(board, 2) returns 0



Calls of `nqueen` that have returned 0 so far: _____

× attacked

Q placed now

Q placed earlier

✗ removed

4 Queens by Hand: Queen at (0, 1)

9 row ____

	0	1	2	3
0				
1				
2				
3				

10 row ____

	0	1	2	3
0				
1				
2				
3				

11 row ____

	0	1	2	3
0				
1				
2				
3				

12 row ____

	0	1	2	3
0				
1				
2				
3				

- `nqueen(board, 4)`: no row is left, it returns ____
- Then every waiting call returns ____
- `nqueen` records on the stack at that moment: ____
- Calls of `nqueen` in total: ____

Final board:

Every row did the same three things: place a queen, call the next row, remove the queen if that call returned 0.

The Backtracking Template

```
int solve(configuration conf) {  
    if (no more choices)           /* base case */  
        return (conf is a goal state);  
    for (all available choices) {  
        try one choice c;  
        ok = solve(conf with choice c made);  
        if (ok)  
            return 1;  
        else  
            unmake choice c;  
    }  
    return 0;                      /* tried all choices */  
}
```

For n-queens:

- configuration:

- no more choices:

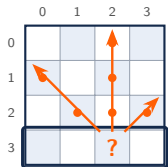
- available choices:

- try choice `c`:

- unmake choice `c`:

Two functions to write: `isSafe` tests one choice, `nqueen` is the template.

In-Class Problem: isSafe



Is (3, 2) safe?
Look up the column
and up both diagonals.

- Queens so far are in rows _____
- Same column: $j == col$
- Diagonal: _____
- Diagonal: _____
- Queens at (0,0) and (1,2):
`isSafe(board, 2, 1)` returns ____

```
int isSafe(int board[N][N], int row, int col) {
    int i, j;
    /* every square (i, j) in the rows above */

    /* queen at (i, j) attacks (row, col)? */

    /* no queen attacks (row, col) */

}
```

In-Class Problem: nqueen

```
int nqueen(int board[N][N], int row) {  
    int c;  
    /* base case: no more rows to fill */  
  
    /* try each column c, left to right */  
  
    /* (row, c) safe? place a queen */
```

main calls _____

```
    /* rows below solved? return 1 */  
  
    /* backtrack: remove the queen */  
  
    /* every column failed */  
  
}
```

All calls share one **board**, stored in _____

Run It: $N = 4$, Then $N = 8$

```
int main() {  
    int board[N][N], i, j;  
    for (i = 0; i < N; i++)    /* empty board */  
        for (j = 0; j < N; j++)  
            board[i][j] = 0;  
    if (nqueen(board, 0) == 1) /* from row 0 */  
        printboard(board);  
    else  
        printf("No solution for N = %d\n", N);  
    return 0;  
}
```

For 8 queens, change one line: _____

$N = 4$ prints

0	1	0	0
0	0	0	1
1	0	0	0
0	0	1	0

$N = 8$, as a board

	0	1	2	3	4	5	6	7
0	Q							
1				Q				
2							Q	
3					Q			
4			Q					
5						Q		
6		Q						
7				Q				

- $N = 4$: the board we found by hand
- $N = 8$: 114 calls of `nqueen`; 113 queens placed, 105 removed
- Boards with one queen in each row,
 $N = 8$: $8^8 = 16,777,216$

Summary & What's Next

Today

- **Problem solving with recursion:** find the smaller problem of the same kind. `movetower` for n discs is `movetower` for $n - 1$, one disc, `movetower` for $n - 1$: $2^n - 1$ moves, at most n records on the stack
- **Backtracking:** make a choice, recurse, and undo the choice when the recursive call returns 0
- **Template:** base case; for each choice: make it, recurse, return 1 on success, otherwise unmake it; return 0
- **n-queens:** `isSafe` tests one square against the rows above; `nqueen` fills one row per call; all calls share one `board`

Next lecture

- File I/O in C: streams, `FILE *`, reading and writing files

Code from today: `hanoi.c`, `nqueens.c`, and the skeletons `hanoi_skel.c`, `nqueens_skel.c`.